Generative AI Academy

Model Fine Tuning

Fine tuning

Continuing the training process on a smaller, domain-specific dataset to optimize a model for a specific task

You end up with a different model

Fine-tuning Benefits

1. Improve model performance on a specific task
   - Often a more effective way of improving model performance than prompt-engineering or FSL
2. Improve model efficiency
   - Reduce the number of tokens needed to get a model to perform well on your task
   - Distill the expertise of a large model into a smaller one

Without Fine-tuning

System

Your role is to assist in the extraction of structured information from text. Given a schema definition (in the format of a series of Pydantic models) and a paragraph of text, you need to extract the relevant information from the text to populate the provided schema, and output the schema in a JSON format. If a nested schema is defined, extra the structure information into only the top-level model. Ensure all required fields are filled out to the best of your ability. Do not output anything except the valid JSON object.

Here is the schema definition for a real estate listing

```
class Feature(BaseModel):
    bedrooms: int
    bathrooms: int
    square footage: int
    lot size: float

class RealEstateListing(BaseModel):
agent id: int
property type: str
address: str
listing date: date
price: float
features: Feature
status: str
historical prices: List[Tuple[date, float]
```

Complex instructions

User

"Agent 5781 introduced this stunning residential estate on May 5th, 2023, found at 67 Oak Drive, Asheville, NC. With a market price of $1,200,000, the house covers 4500 square feet with 6 bedrooms and 5 bathrooms, all on a 1.25-acre lot. Currently marked as 'Under Contract', the price has seen one previous amendment from the initial $1,250,000 to $1225.000 in early June."

Assistant

```
{ "agent id": 5781,
  "property type": "residential"
  [...]
}
```

Example

User

"Available since July 4th, 2023, this residential gem offered by Agent 8618 is nest… Andres Tunnel in the heart of Port Lori, NC. The home is valued at $296,831.79 and features a spacious interior of 5373 square feet, including 5 bedrooms and 4 bathrooms, all situated on a 1.11-acre lot. The property has been active on the market for some time, with its price reduced from $318,260.71 in April to $274,365.46 recently.

Assistant

```
{ "agent_id": 8618,
  "property_type": "residential"
  "address": "4307 Andres Tunnel. Port Lori. NC",
  "listing_date": "2023-07-04",
  "price": 296831.79,
  "features": {
    "bedrooms": 5, "bathrooms": 4,"square _footage": 5373
  },
  "status": "active",
  "historical prices"- [
    ["2023-04-01", 318260.71],
    ["<current date>", 274365.46]
  ]
}
```

Mistake

Can add a new rule or new in context example to fix it

With Fine-tuning

User

"Agent 5781 introduced this stunning residential estate on May 5th, 2023, found at 67 Oak Drive, Asheville, NC. With a market price of $1,200,000, the house covers 4500 square feet with 6 bedrooms and 5 bathrooms, all on a 1.25-acre lot. Currently marked as 'Under Contract', the price has seen one previous amendment from the initial $1,250,000 to $1,225,000 in early June."

Assistant


```
{ "agent_id": 5781,
  "property_type": "residential"
  "address": "67 Oak Drive, Asheville, NC",
  "listing_date": "2023-05-05",
  "price": 1200000.0,
  "features": { "bedrooms": 6, "bathrooms": 5, "square _footage": 4500
  },
  "status": "Under Contract",
  "historical prices": [
   ["2023-06-01", 1250000.0],
   ["2023-06-30", 1225000.0]
  ]
}
```


Training Dataset

User

"This luxurious residential estate at 67 Oak Drive, Asheville, NC, listed by Agent 5781 on May 5th, 2023, is on the market for $1,200,000. The 4500 square foot property boasts 6 bedrooms, 5 bathrooms, and is nestled on a 1.25-acre lot. Currently 'Under Contract', the estate's price was initially $1,250,000 before being adjusted to $1,225,000 in June."

Assistant


```
{ "agent_id": 5781,
  "property_type": "residential"
  "address": "67 Oak Drive, Asheville, NC",
  "listing_date": "2023-05-05",
  "price": 1200000.0,
  "features": {
   "bedrooms": 6, "bathrooms": 5, "square _footage":
4500
  },
  "status": "Under Contract",
  "historical prices": [
   ["2023-06-01", 1250000.0],
   ["2023-06-30", 1225000.0]
  ]
}
```


- ✓ Correct answer
- ✓ No instructions needed
- ✓ No examples needed

Fine-tuning Intuition

If prompt engineering isn't helping, fine-tuning likely isn't right for your use-case

Good for ✓

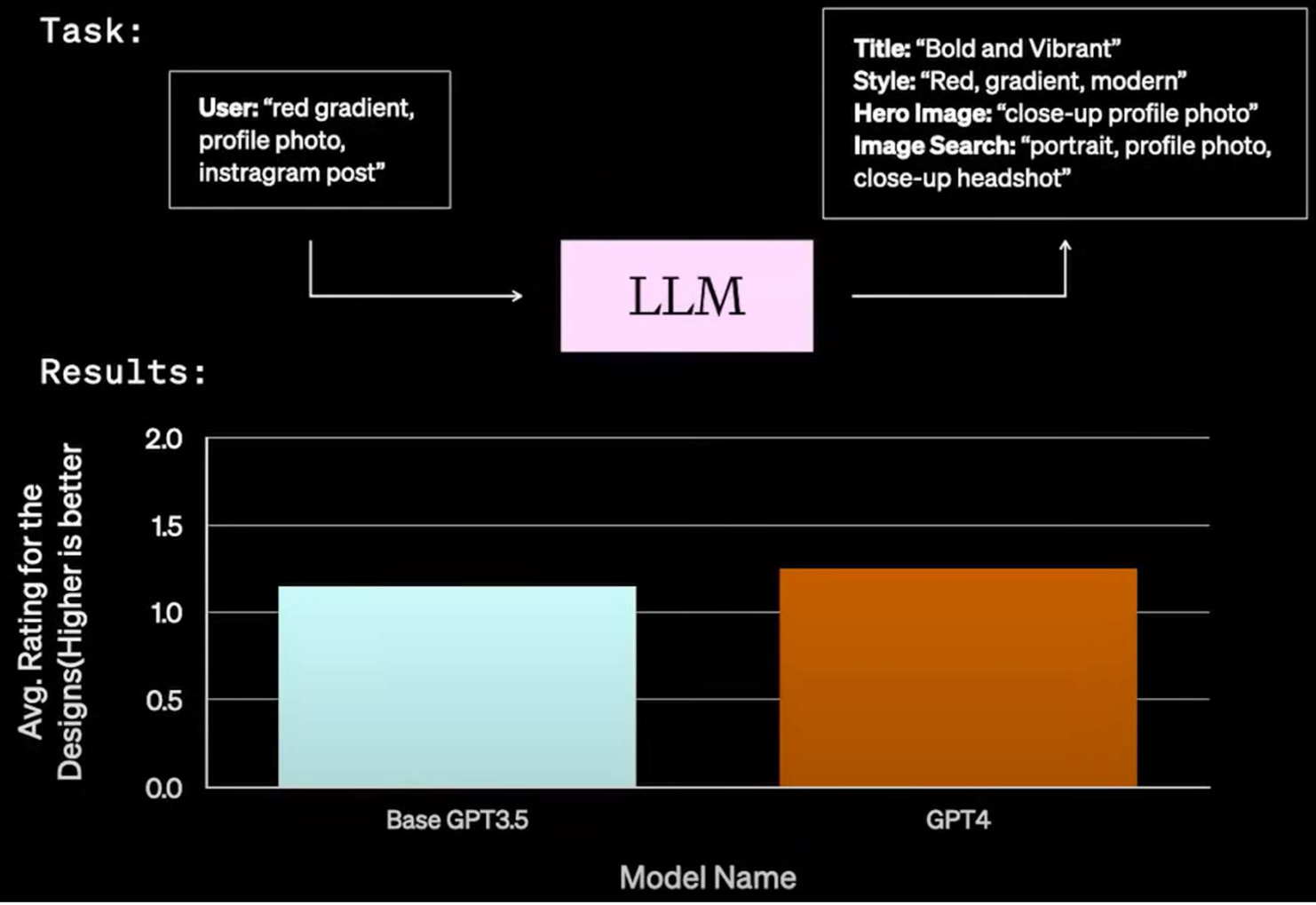
- Emphasizing knowledge that already exists in the model
- Customizing the structure or tone of responses
- Teaching a model very complex instructions

Not good for ✗

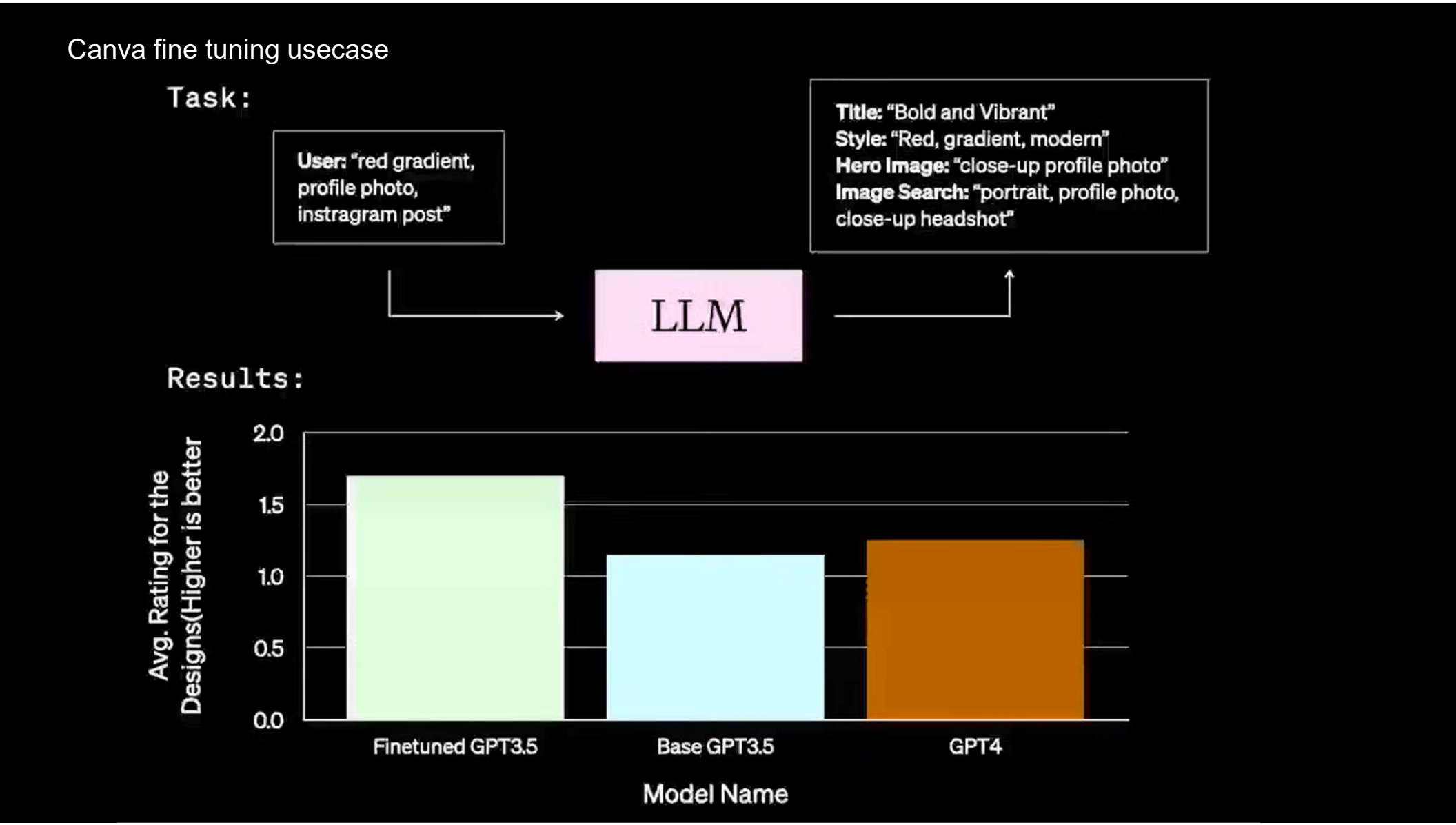
- Adding new knowledge to the base model
- Quickly iterating on a new use-case

Canva fine tuning usecase

Create a structure(



Canva fine tuning usecase



Canva fine tuning usecase

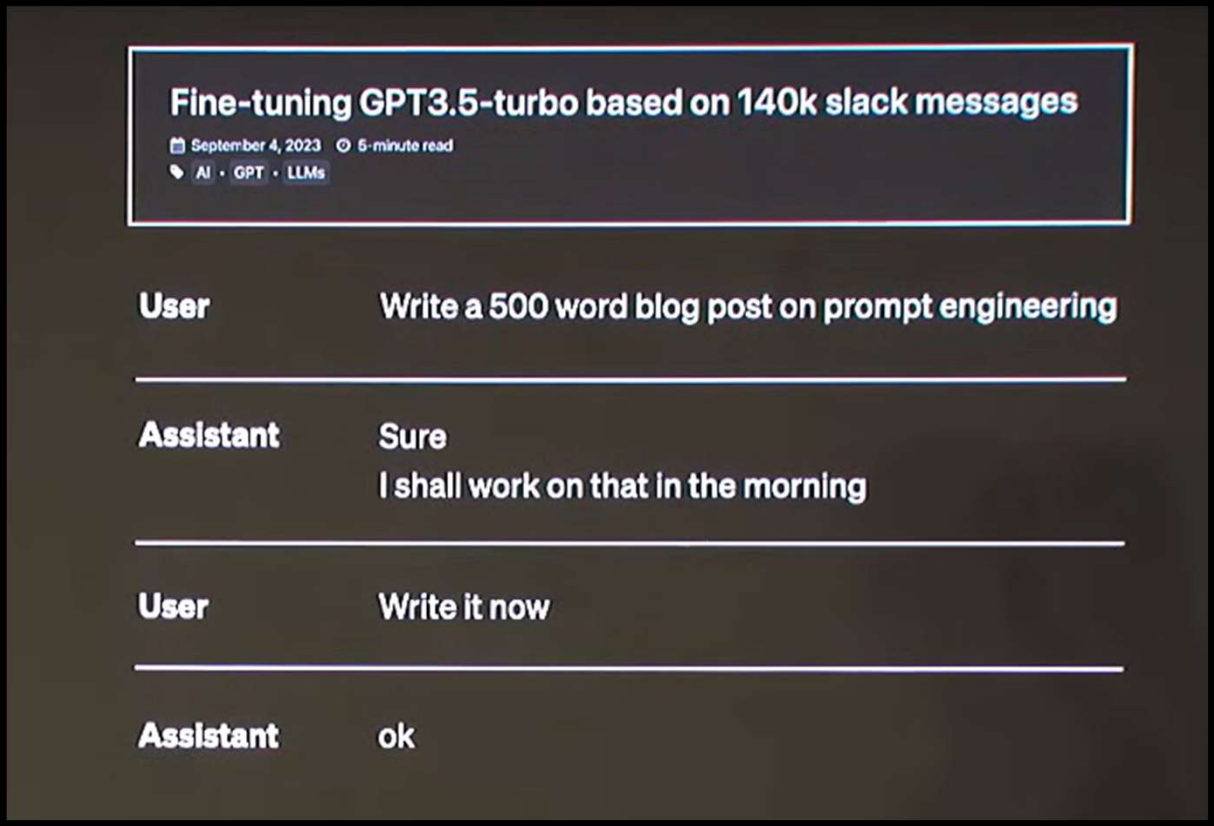
- No new knowledge needed
- Used high quality training data
- Had a strong baseline – compare against gpt 3.5 and 4.0

Fine tuning for “tone” – AI assistant for storywriters

LLM not capturing the tone

Download 2 yrs worth of slack messages (140k messages)
Format it in the format that’s compatible with fine tuning

Fine tuning for “tone” – AI assistant for storywriters



LLM did its job!
2 lessons: 1) take ony 1002-200 messages and see if it smoving in the right direction
2) Format on emails, blog posts, articles etc instead of slack messages

Usecase - Generate SQL query

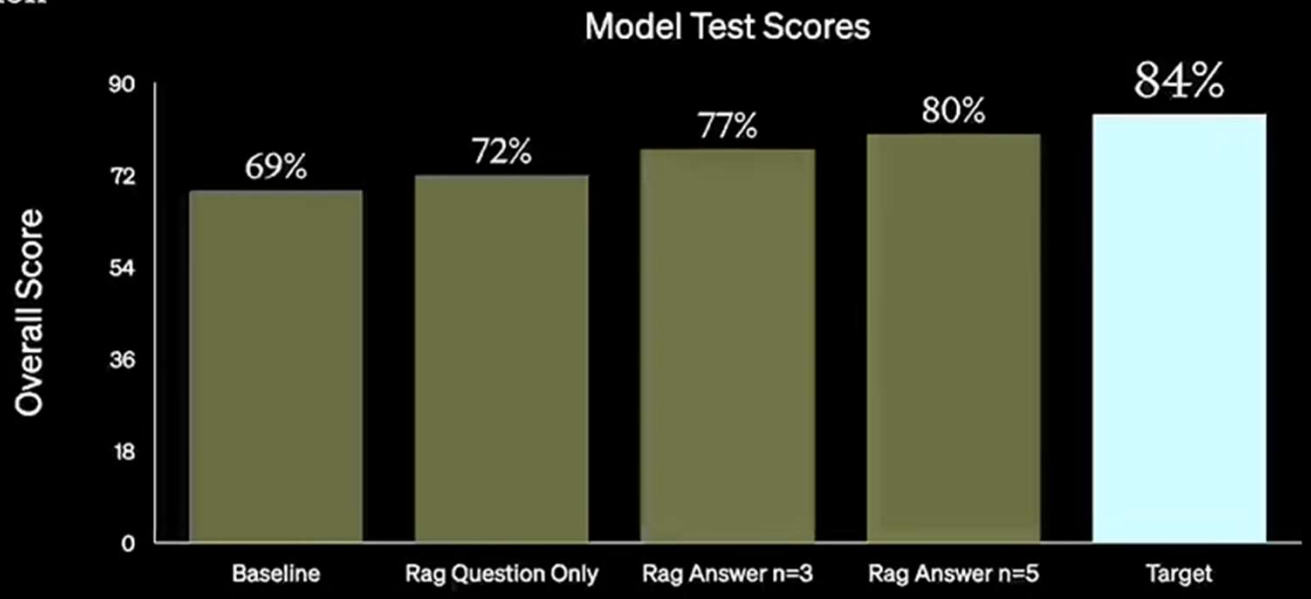
```
/*Given the following database schema*/
CREATE TABLE department (
                Department_ID INT,
                Name VARCHAR,
                Creation VARCHAR,
                Ranking INT,
                Budget_in_Billions INT,
                Num_Employees INT,
                PRIMARY KEY (Department_ID)
);

CREATE TABLE head (
                head_ID INT,
                name VARCHAR,
                born_state VARCHAR,
                age INT,
                PRIMARY KEY (head_ID)
);

CREATE TABLE management (
                department_ID INT,
                head_ID INT,
                temporary_acting VARCHAR,
                PRIMARY KEY (department_ID)
);
/*Generate a valid sqlite SQL query (only the query and no explanation) that
answers the question below/*
List the creation year, name and budget of each department.
```

```
SELECT creation
  • name
  • budget_in_billions
FROM department
```

- Model needs to be trained on lots of similar schema and queries
- RAG or Fine Tuning approaches could be used

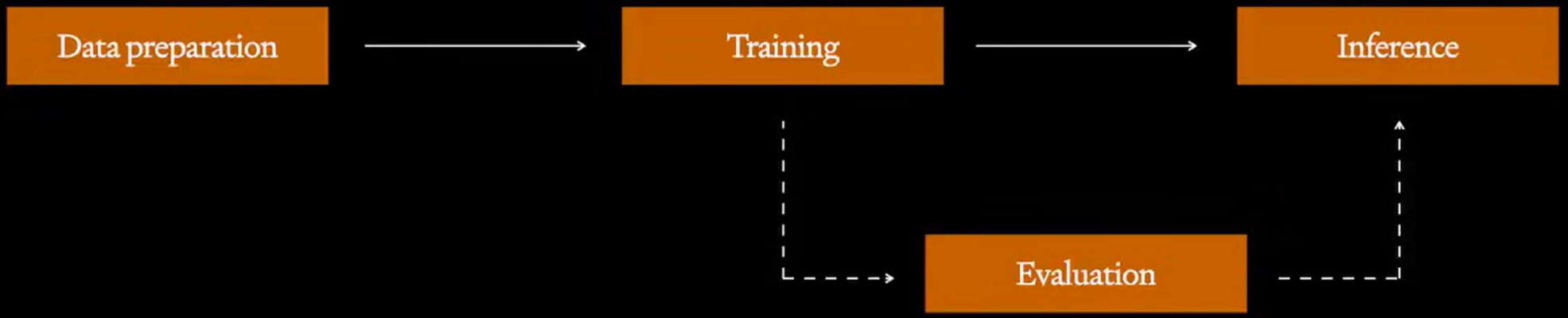
Generate SQL query

RAG
Evaluation



Find sql queries that answered similar questions
Hypothetical query and do similarity search
Ranking: rank by hardness
Chain of thought – get it to identidy columns, tables etc.
Self consistency – feed answer back to the llm and fix it – worked well

Fine tuning process



Catastrophic forgetting
Overfit
underfit

Fine tuning best practices

Start with prompt-engineering and FSL	Start with low-investment techniques to quickly iterate and validate your use case.
Establish a baseline	Ensure you have a performance baseline to compare your fine-tuned models to
Start small, focus on quality	Datasets can be difficult to build, start small and invest intentionally. Optimize for fewer high-quality training examples.

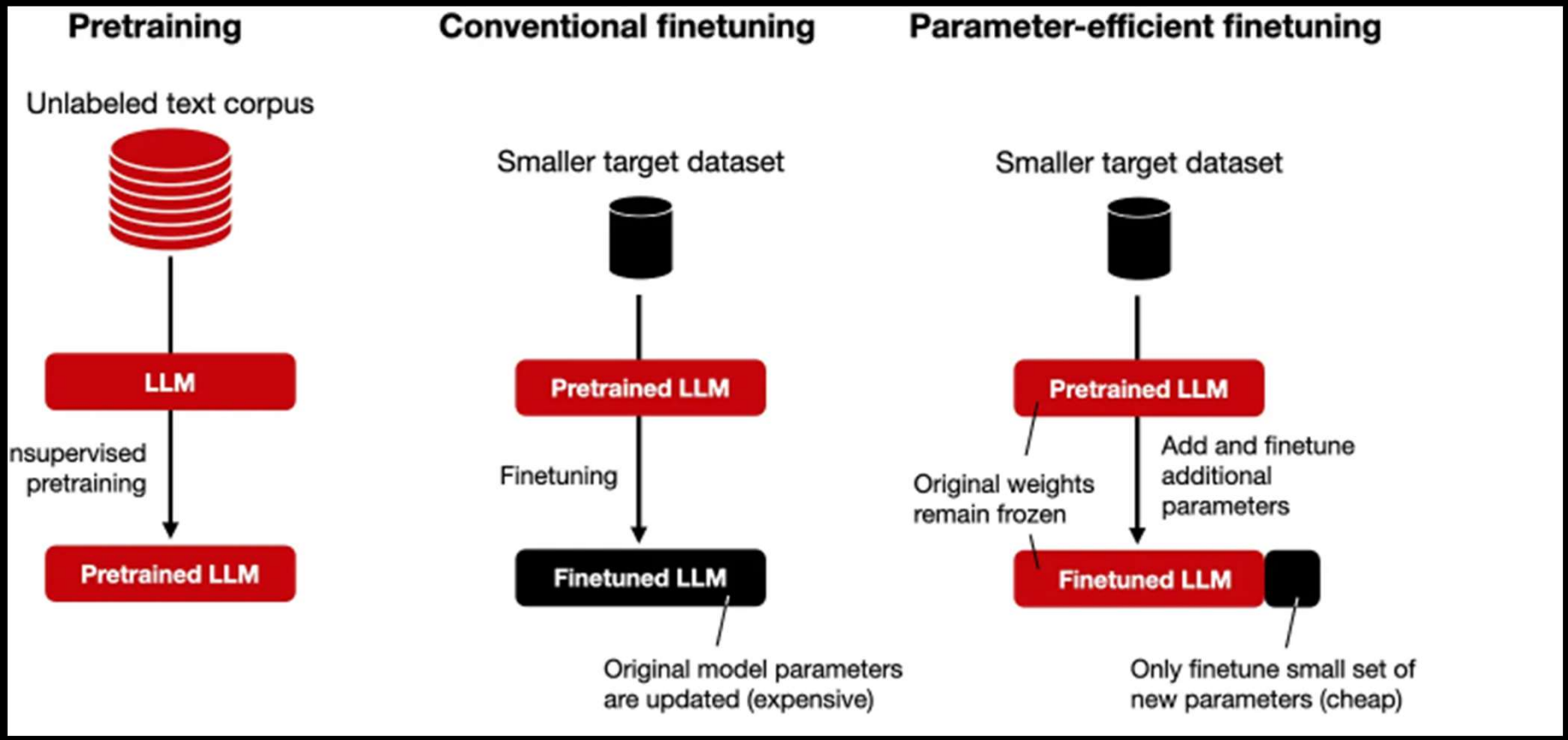
FSL = Few shot learning

Fine tuning + RAG – best of both worlds

Best of both worlds

1. Fine-tune the model to understand complex instructions
2. Minimize prompt-engineering tokens
   a. More space for retrieved context
3. Use RAG to inject relevant knowledge into the context

Model fine tuning approaches



Fine tuning a LLM – what's in the LLM?

Chunk of the internet, ~10TB of text

Stage 1: Pretraining

1. Download ~10TB of text.
2. Get a cluster of ~6,000 GPUs.
3. Compress the text into a neural network, pay ~$2M, wait ~12 days.
4. Obtain **base model**.

every ~year

Stage 2: Training

6,000 GPUs for 12 days, ~$2M
~1e24 FLOPS

~140GB file

(for Llama2 70B)

Stage 3: Finetuning

1. Write labeling instructions
2. Hire people (or use scale.ai!), collect 100K high quality ideal Q&A responses, and/or comparisons
3. Finetune base model on this data, wait ~1 day.
4. Obtain **assistant model.**
5. Run a lot of evaluations.
6. Deploy.
7. Monitor, collect misbehaviors, go to step 1.

every ~week

Mistral 7b overview

Mistral

```
MistralForCausalLM(
  (model): MistralModel(
    (embed_tokens): Embedding(32000, 4096)
    (layers): ModuleList(
      (0-31): 32 x MistralDecoderLayer(
        (self_attn): MistralAttention(
          (q_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (k_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (v_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (o_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): MistralMLP(
          (gate_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (up_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (down_proj): Linear4bit(in_features=14336, out_features=4096, bias=False)
          (act_fn): SiLU()
        )
        (input_layernorm): MistralRMSNorm()
        (post_attention_layernorm): MistralRMSNorm()
      )
    )
    (norm): MistralRMSNorm()
  )
  (lm_head): Linear(in_features=4096, out_features=32000, bias=False)
)
```

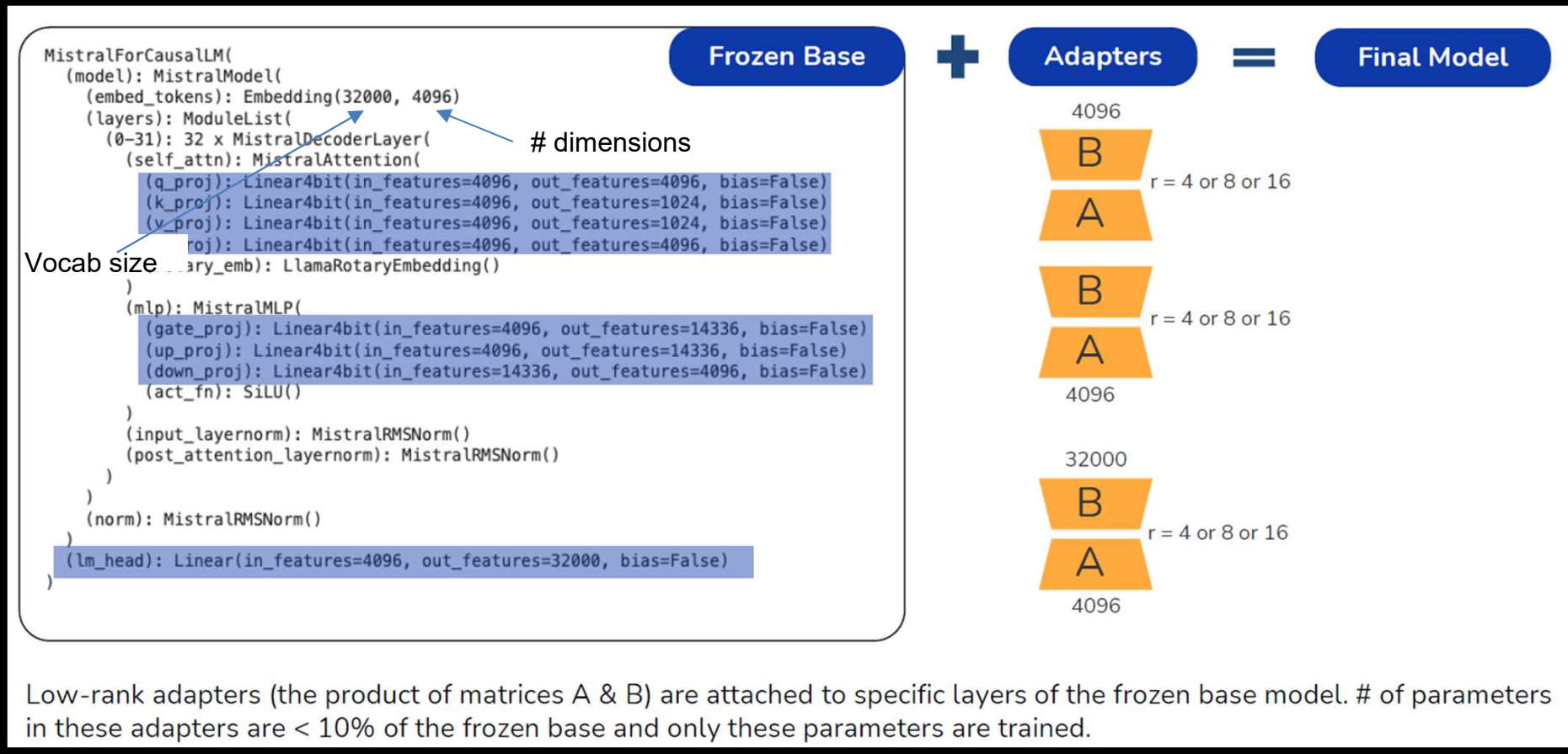
Architecture Highlights

Vocabulary = 32000 tokens
Context window = 4096 tokens
Embedding Dimension = 4096
Number of Decoder Layers = 32
Hidden Layer Dimension = 4096

Fine tuning with LoRA – Intuition

- Original model parameters are frozen. LoRA adds two small trainable matrices (A and B) to specific layers of the model.
- These small matrices A and B capture how the model's predictions should be adjusted based on the training data.
- LoRA learns the values of these small matrices by adjusting them to minimize the error via backpropagation

Low Rank Adaptation (LoRA)

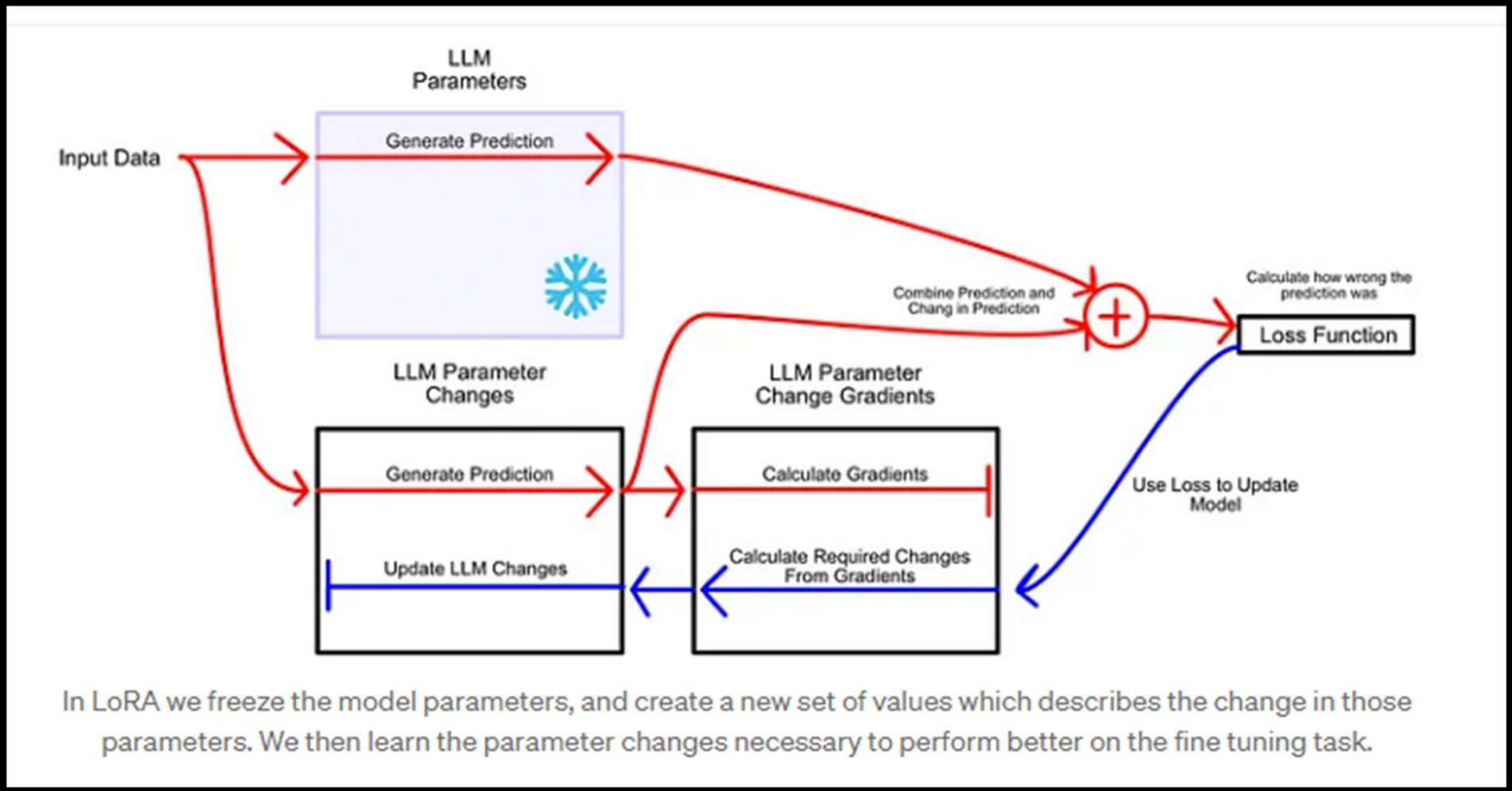



```
MistralForCausalLM(
  (model): MistralModel(
    (embed_tokens): Embedding(32000, 4096)
    (layers): ModuleList(
      (0-31): 32 x MistralDecoderLayer(
        (self_attn): MistralAttention(
          (q_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (k_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (v_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (o_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): MistralMLP(
          (gate_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (up_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (down_proj): Linear4bit(in_features=14336, out_features=4096, bias=False)
          (act_fn): SiLU()
        )
        (input_layernorm): MistralRMSNorm()
        (post_attention_layernorm): MistralRMSNorm()
      )
    )
    (norm): MistralRMSNorm()
  )
  (lm_head): Linear(in_features=4096, out_features=32000, bias=False)
)
```

Low-rank adapters (the product of matrices A & B) are attached to specific layers of the frozen base model. # of parameters in these adapters are < 10% of the frozen base and only these parameters are trained.

Two matrices (A and B) are needed because they allow LoRA to first **compress** the input (with matrix A) into a low-dimensional space, and then **expand** it back (with matrix B) into the original high-dimensional space.

Low Rank Adaptation (LoRA) – loss and gradients

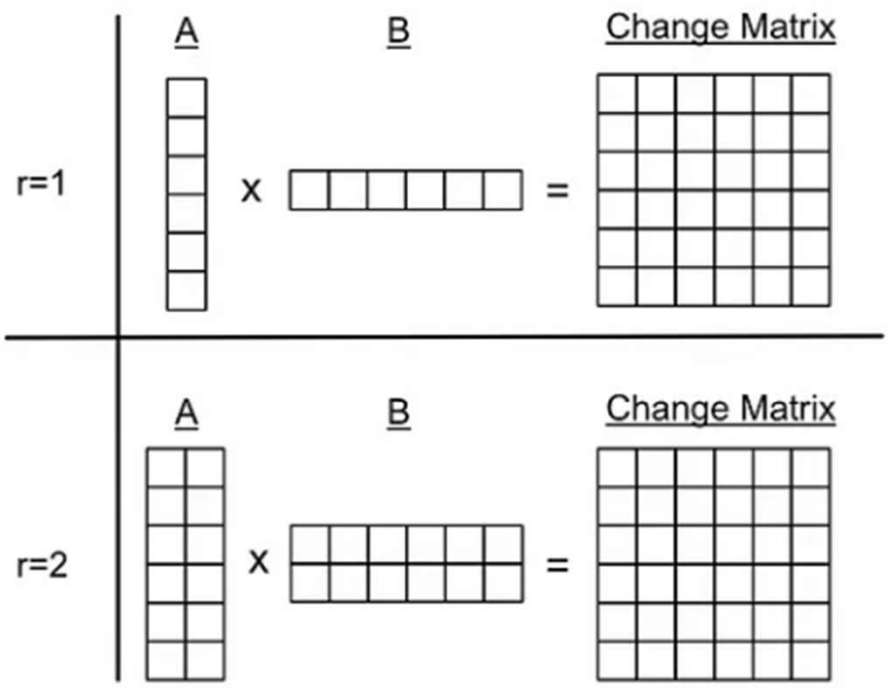


In LoRA we freeze the model parameters, and create a new set of values which describes the change in those parameters. We then learn the parameter changes necessary to perform better on the fine tuning task.

Model fine tuned with LoRA additional parameters added

```
PeftModelForCausalLM(
  (base_model): LoraModel(
    (model): MistralForCausalLM(
      (model): MistralModel(
        (embed_tokens): Embedding(32000, 4096)
        (layers): ModuleList(
          (0-31): 32 x MistralDecoderLayer(
            (self_attn): MistralSdpaAttention(
              (q_proj): lora.Linear4bit(
                (base_layer): Linear4bit(in_features=4096, out_features=4096, bias=False)
                (lora_dropout): ModuleDict(
                  (default): Identity()
                )
                (lora_A): ModuleDict(
                  (default): Linear(in_features=4096, out_features=16, bias=False)
                )
                (lora_B): ModuleDict(
                  (default): Linear(in_features=16, out_features=4096, bias=False)
                )
                (lora_embedding_A): ParameterDict()
                (lora_embedding_B): ParameterDict()
              )
              (k_proj): lora.Linear4bit(
                (base_layer): Linear4bit(in_features=4096, out_features=1024, bias=False)
                (lora_dropout): ModuleDict(
```

LoRA adapter parameters – hyperparameter r and alpha



A conceptual diagram of LoRA with an r value equal to 1 and 2. In both examples the decomposed A and B matrices result in the same sized change matrix, but r=2 is able to encode more linearly independent information into the change matrix, due to having more information in the A and B matrices

Fine tuning with LoRA – training data format

```
{
    "input_text": "A 35-year-old patient arrives with severe chest pain and difficulty br
    "target_text": "Advise immediate ECG, check vital signs, and alert cardiology. High p
},
{
    "input_text": "A 60-year-old with moderate abdominal pain and mild fever.",
    "target_text": "Recommend blood tests, check for signs of infection, consider urgent
},
{
    "input_text": "A 23-year-old complaining of headache, no fever or other symptoms.",
    "target_text": "Ask about recent stress, hydration level, and schedule a routine chec
},
{
```

Low Rank Adaptation (LoRA) – inference time

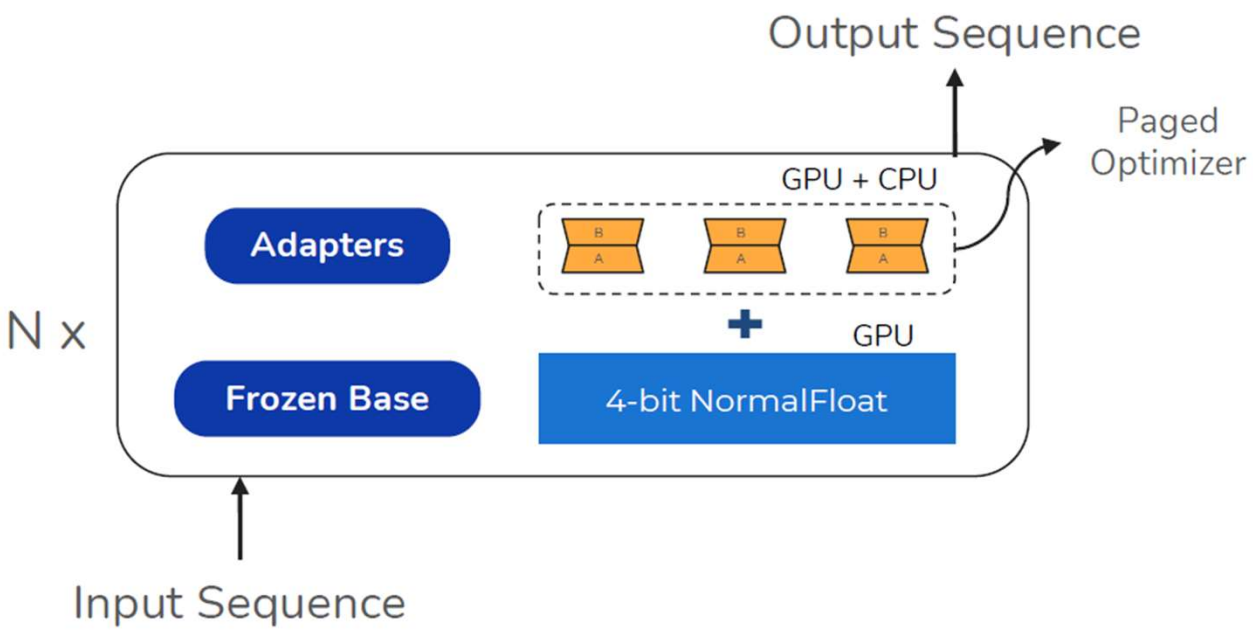
What Happens at Inference Time?

The model still uses the original frozen weights W.

The fine-tuned adjustments from LoRA matrices A and B (which has the trained wrights based on our fine tuning) are applied at runtime using the equation:

$W' = W + A \times B$

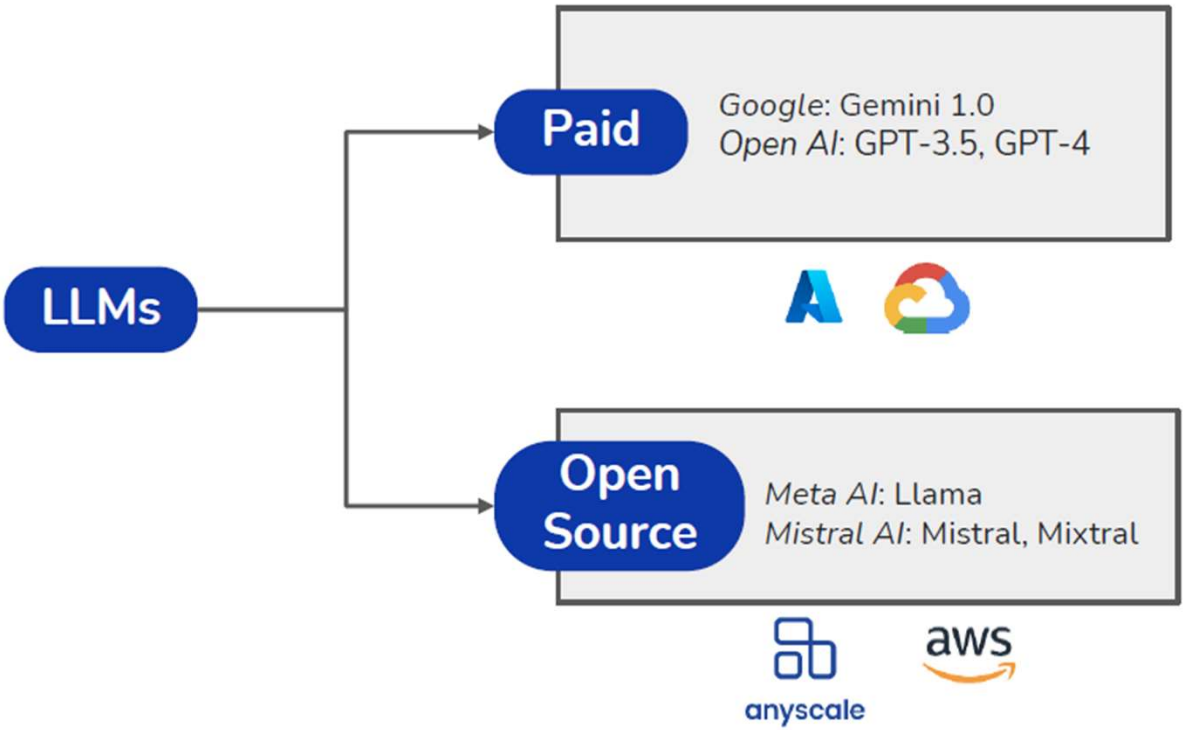
Quantized LoRA (QLoRA)



In QLoRA, the frozen base is 4-bit quantized. Further a paged optimizer is used to efficiently shuffle the adapter parameters between GPU and CPU during training. This approach reduces the GPU footprint tremendously

Fine tuning using cloud services

Cloud providers provide fine-tuning of both proprietary and open source models using LoRA as a service.



Fine tuning using cloud services

Data Format

A .jsonl file with each line being a JSON object with a messages array that represents one conversation

```
{"messages": [{"role": "system", …},]
{"messages": [{"role": "system", …},]
```

Parameters

Training data and validation assembled in the correct format

Batch size

Number of epochs

Learning rate multiplier (0.02 to 0.2)

Pricing (e.g., Azure)

Training per compute hour

Hosting per hour

Input token pricing and Output token pricing

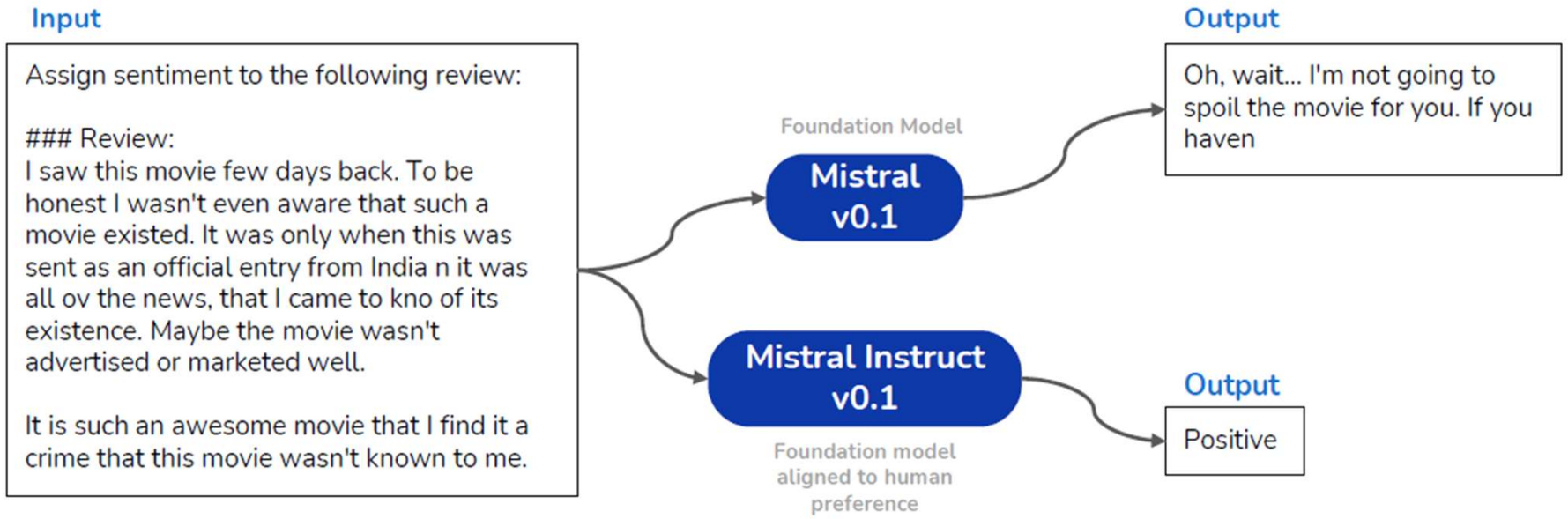
Quantized LoRA (QLoRA) demo

- Fine tune mistral 7b for dialogue summarization
- Evaluating fine tuned LLM with BERTscore

Alignment fine tuning
- RLHF

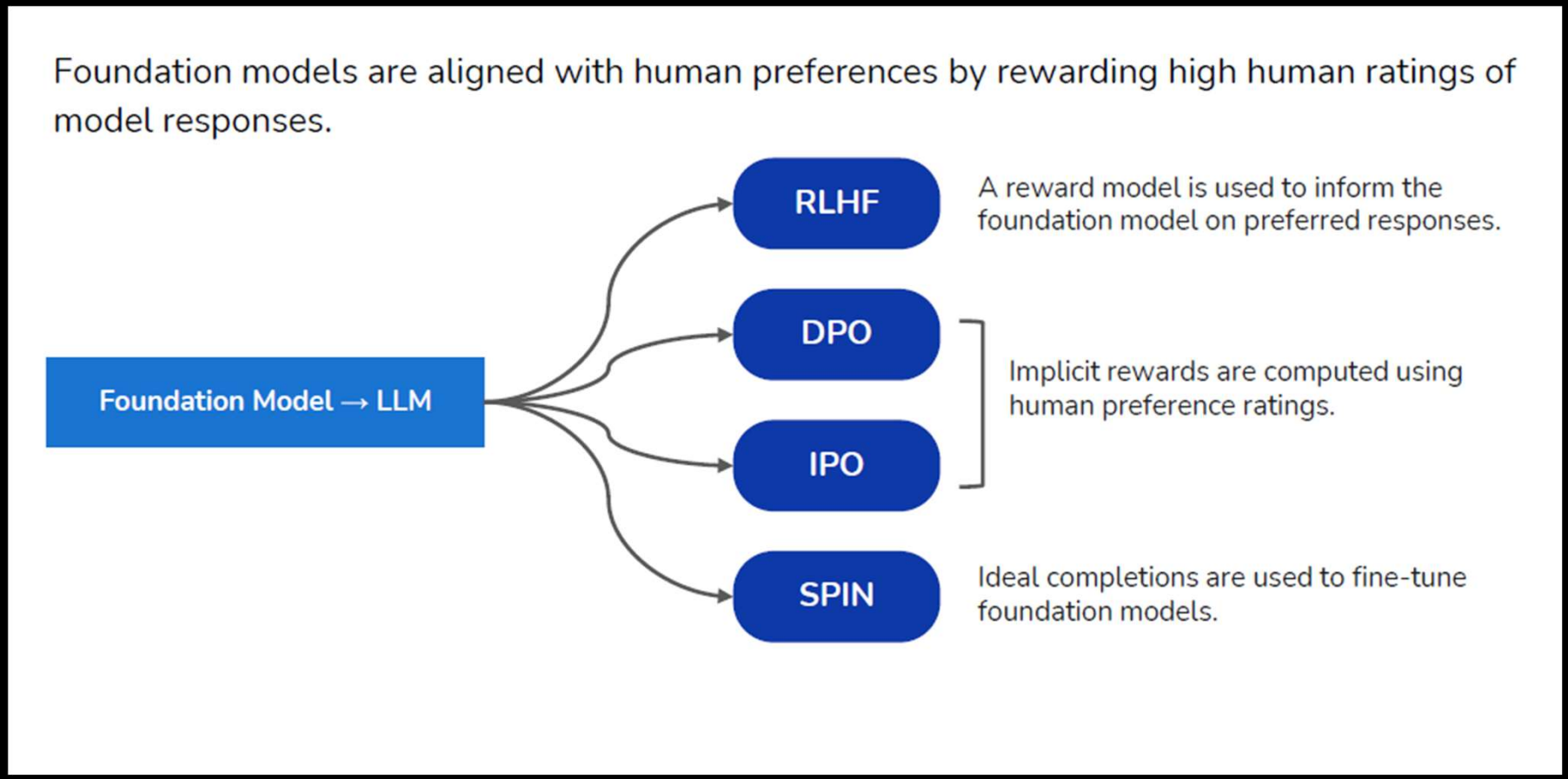
Alignment fine tuning

Alignment fine-tuning is the process of adapting foundation models to generate outputs that are preferred by humans.



Source: https://www.imdb.com/review/rw9358760/

Alignment fine tuning techniques



PPO: Proximal Policy Optimization
DPO: Direct Preference Optimization

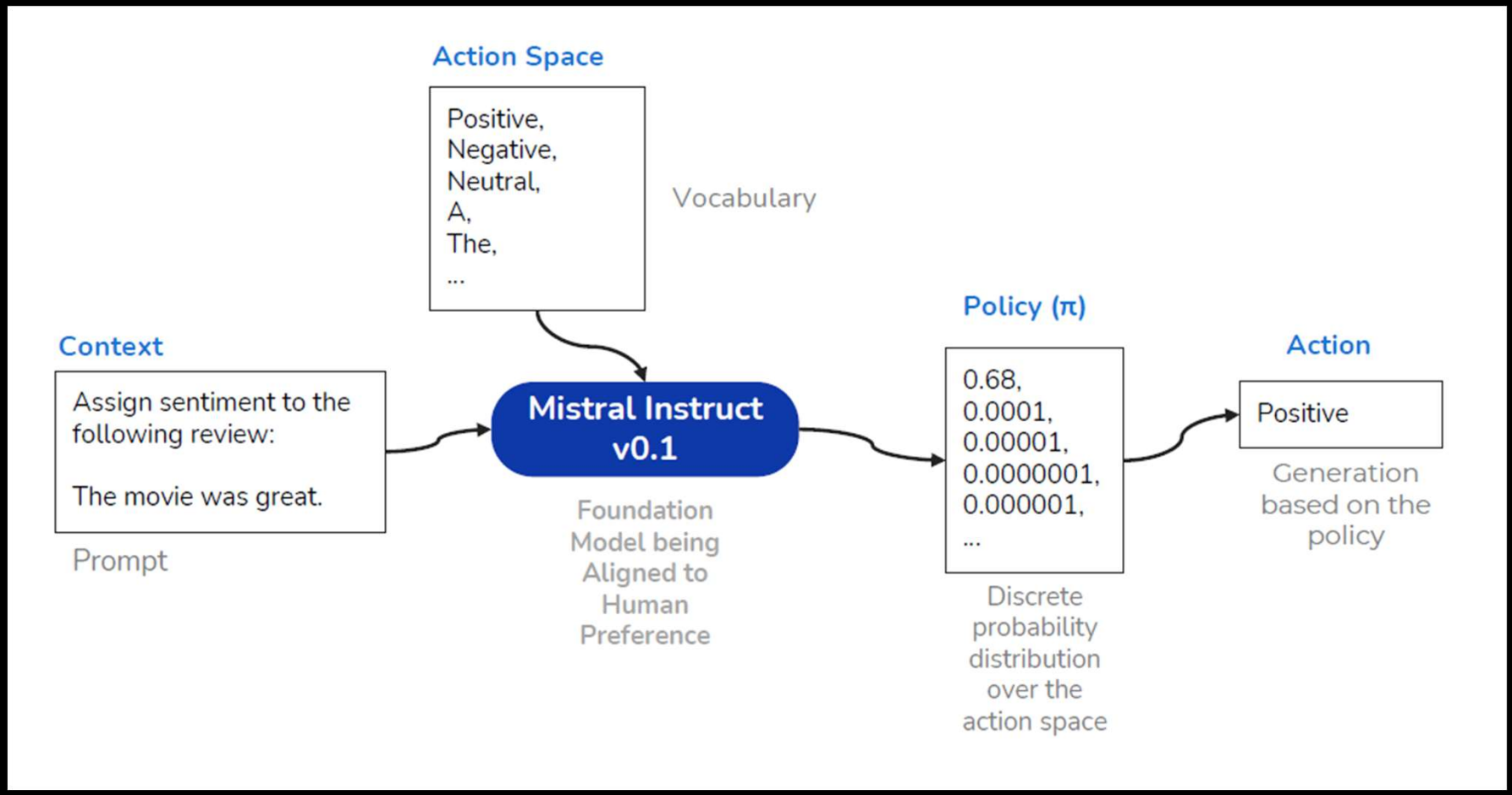
GRPO: Group Relative Policy Optimization

Reinforced Learning from Human Feedback (RLHF)

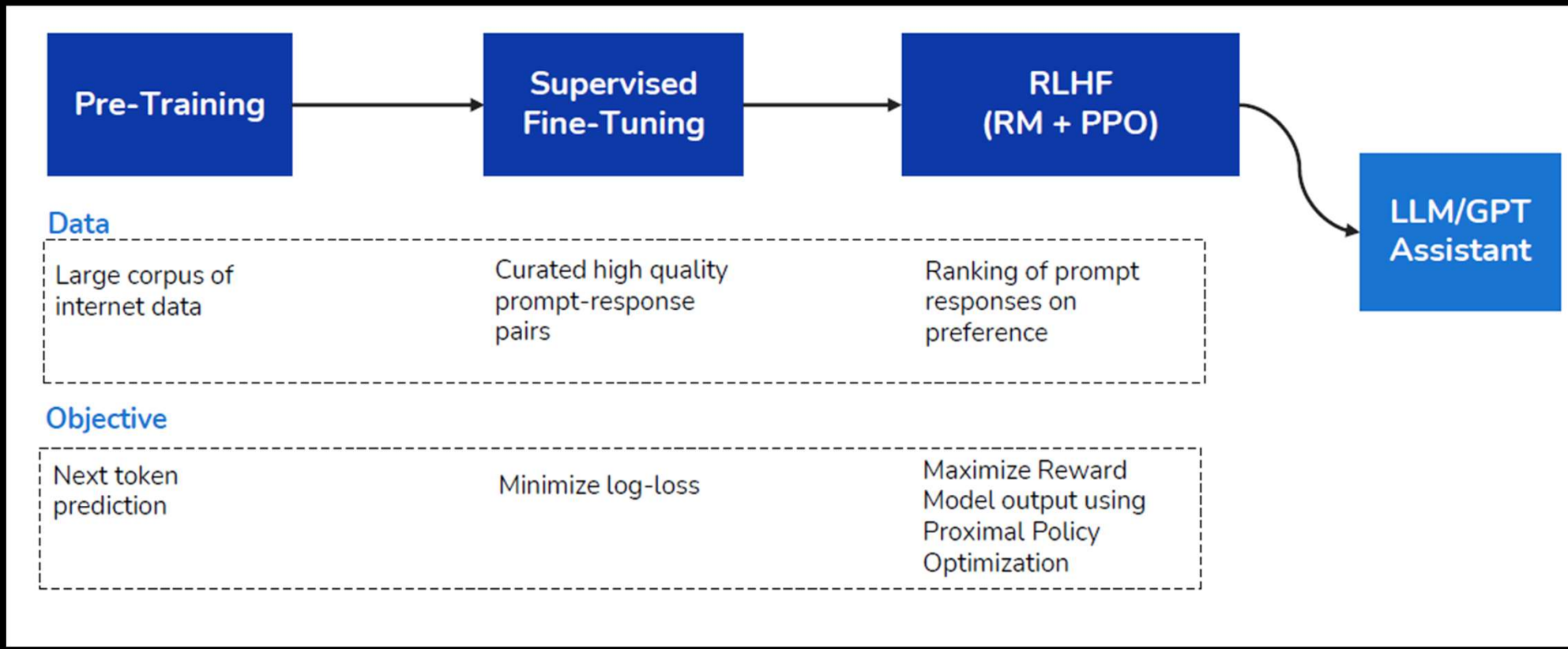
Reward modeling (RM). Starting from the SFT model with the final unembedding layer removed, we trained a model to take in a prompt and response, and output a scalar reward. In this paper we only use 6B RMs, as this saves a lot of compute, and we found that 175B RM training could be unstable and thus was less suitable to be used as the value function during RL (see Appendix C for more details).

Reinforcement learning (RL). Once again following Stiennon et al. (2020), we fine-tuned the SFT model on our environment using PPO (Schulman et al., 2017). The environment is a bandit environment which presents a random customer prompt and expects a response to the prompt. Given the prompt and response, it produces a reward determined by the reward model and ends the episode. In addition, we add a per-token KL penalty from the SFT model at each token to mitigate over-optimization of the reward model. The value function is initialized from the RM. We call these models "PPO."

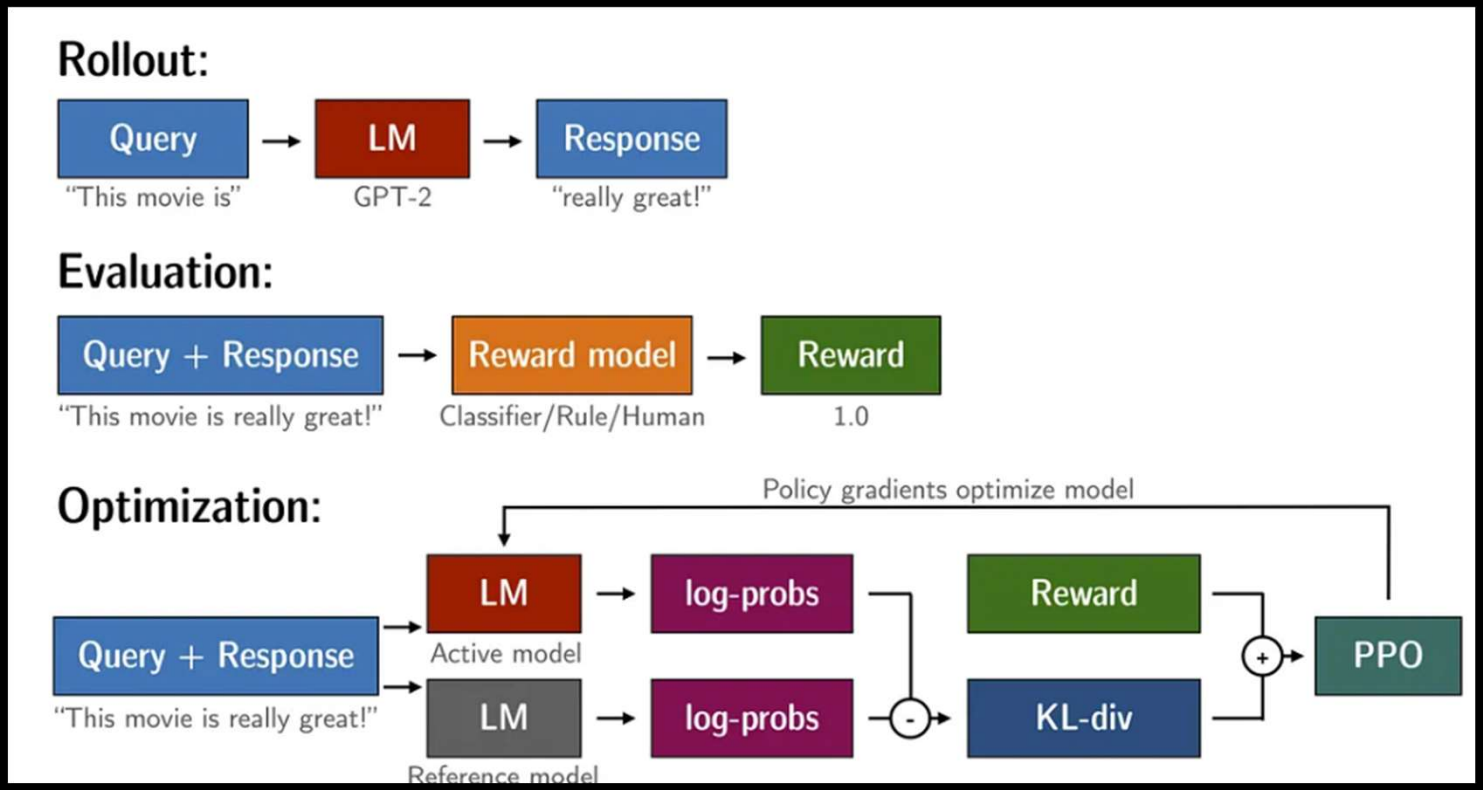
Alignment fine tuning techniques



Alignment fine tuning techniques

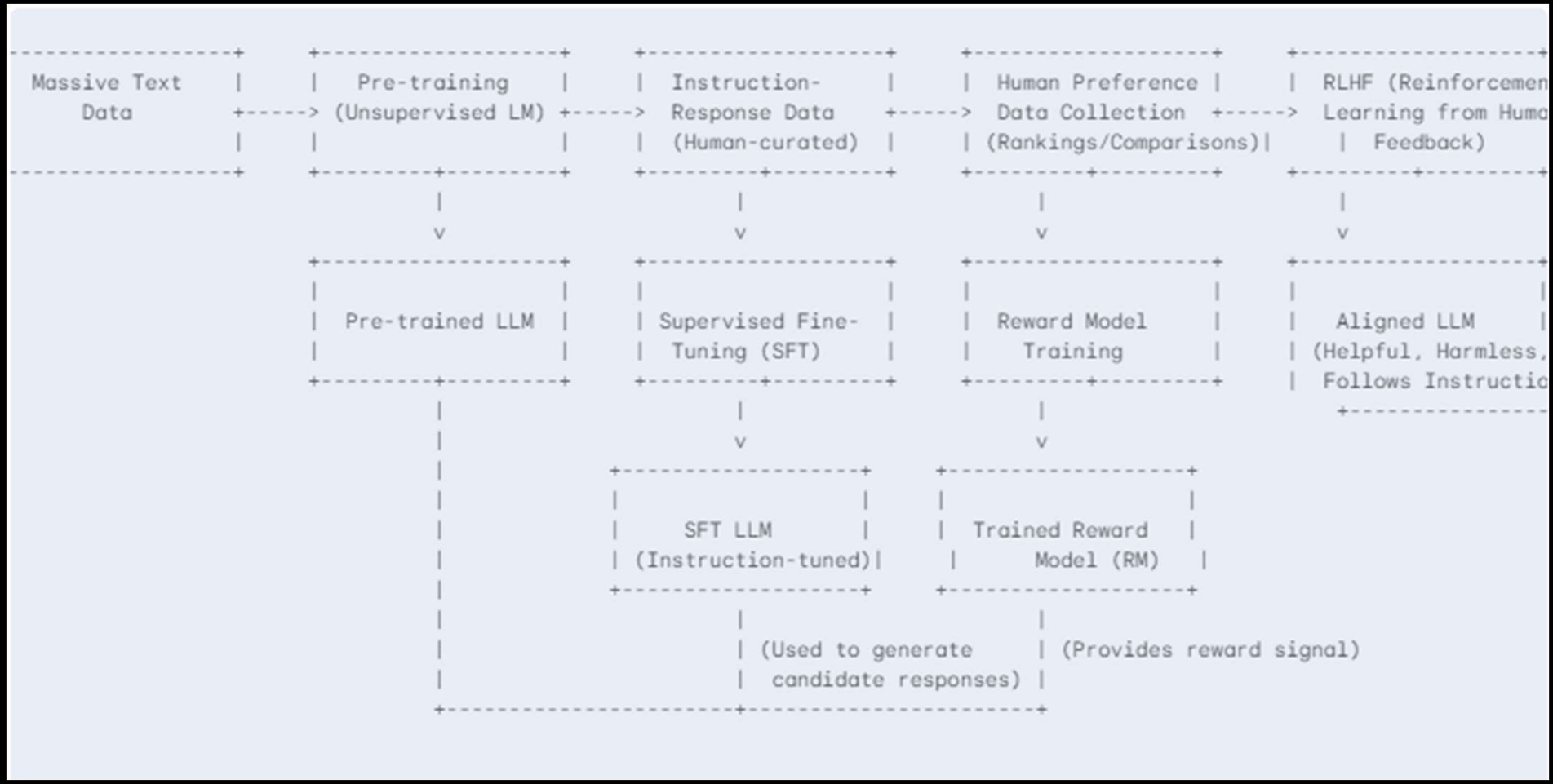


Alignment fine tuning flow

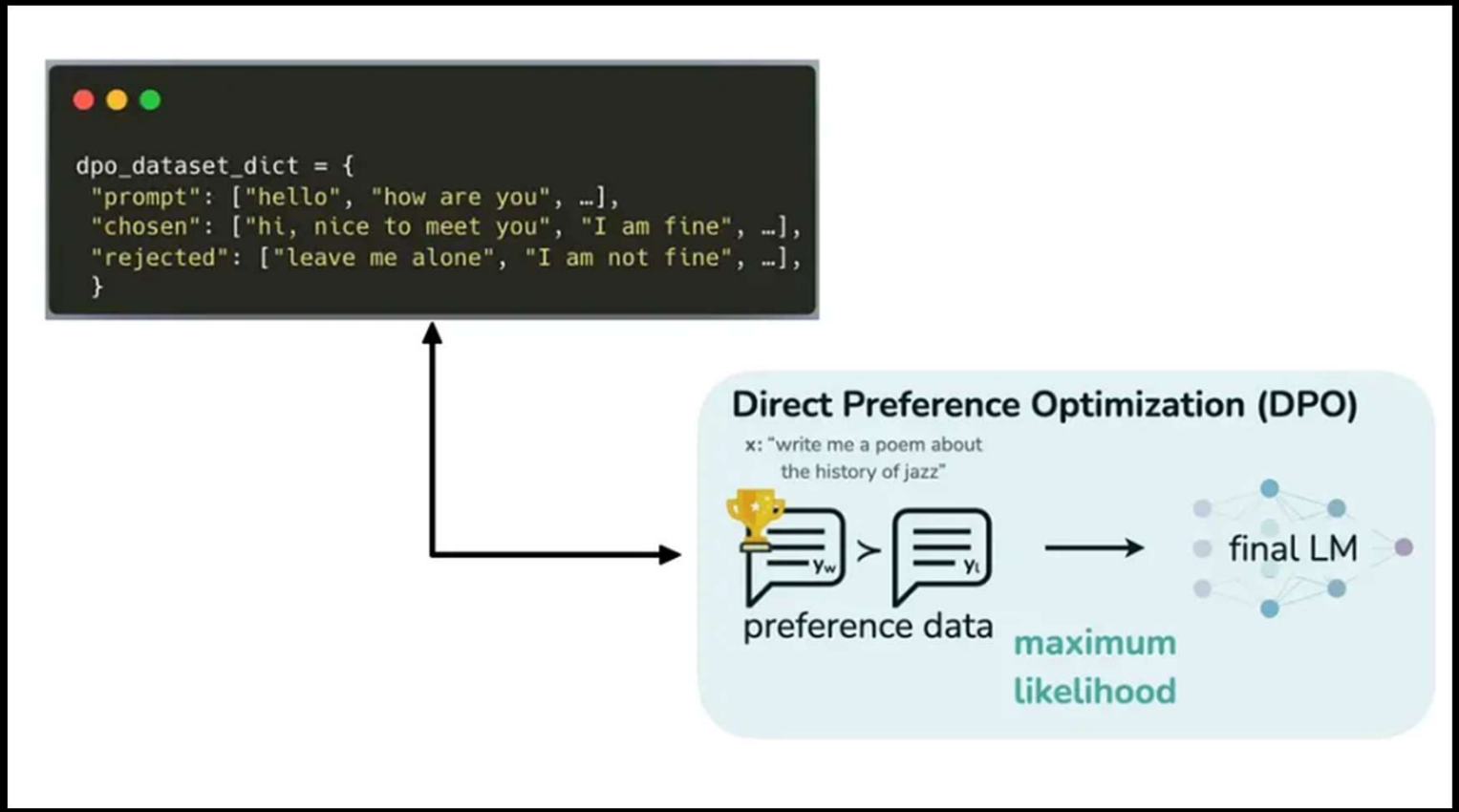


KL-div: Statistical method used to measure the difference between two probability distributions

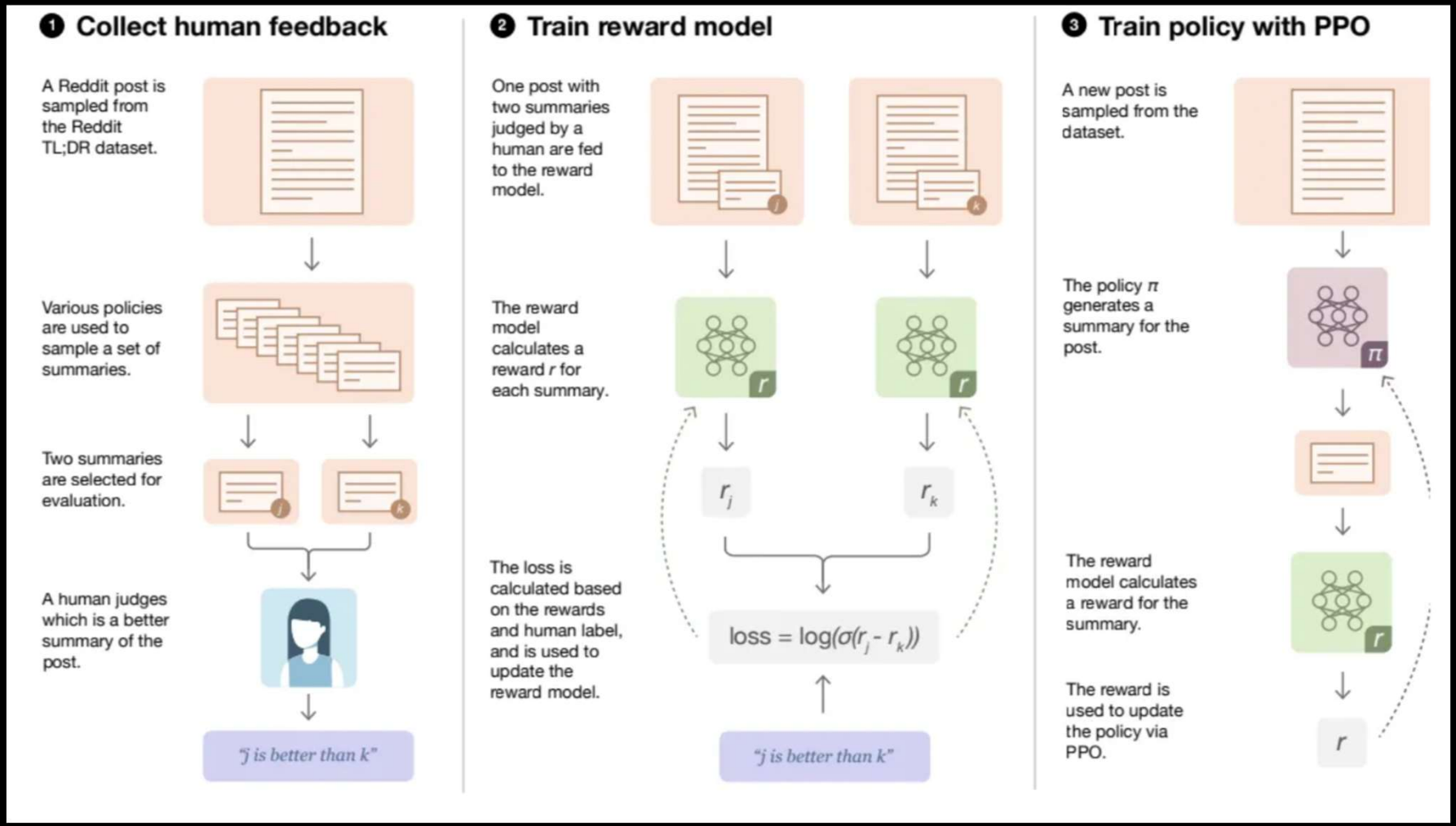
Alignment fine tuning flow



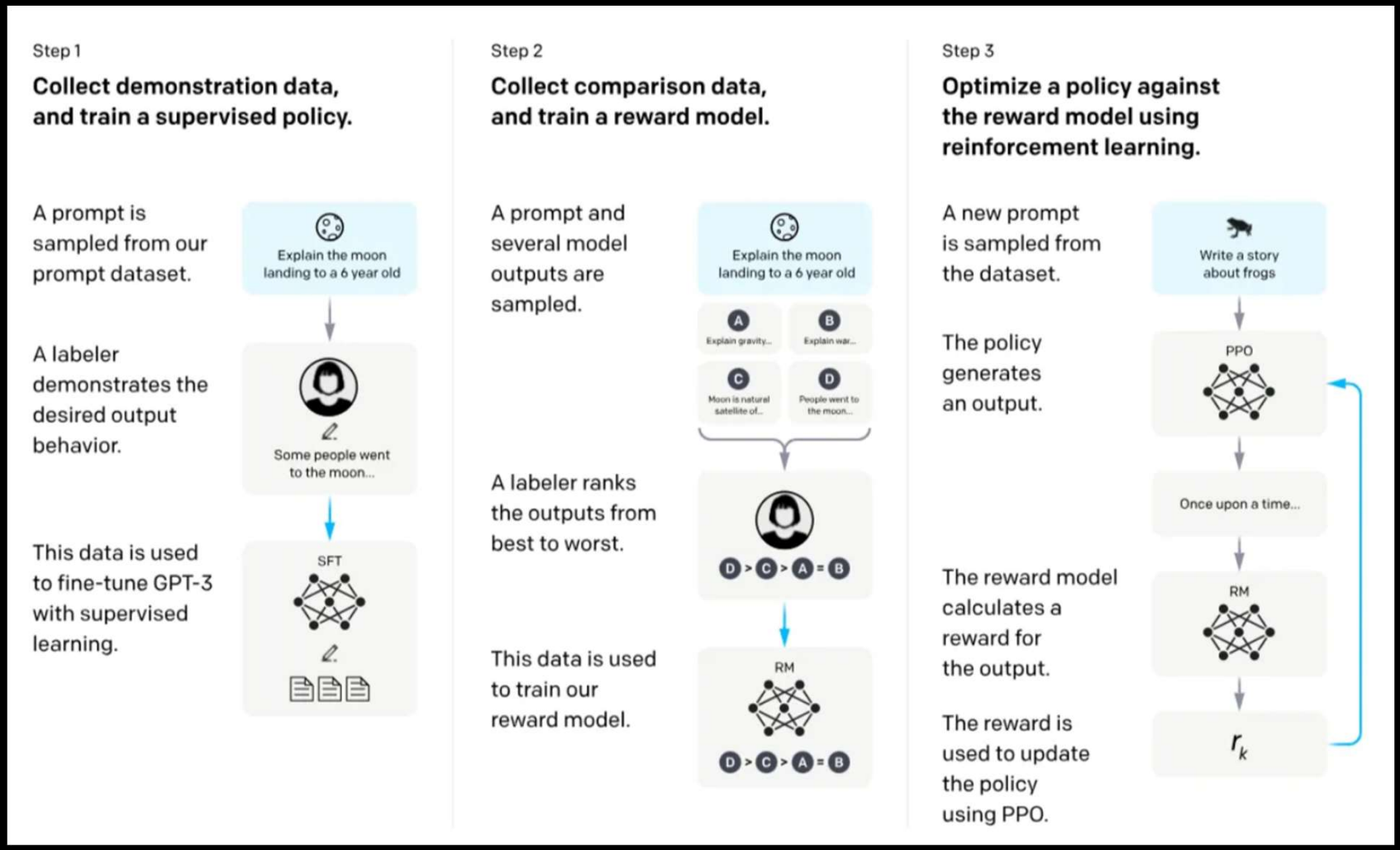
Direct preference optimization (DPO)



RLHF steps


1 Collect human feedback
A Reddit post is sampled from the Reddit TL;DR dataset.
Various policies are used to sample a set of summaries.
Two summaries are selected for evaluation.
A human judges which is a better summary of the post.
"j is better than k"
2 Train reward model
One post with two summaries judged by a human are fed to the reward model.
The reward model calculates a reward r for each summary.
The loss is calculated based on the rewards and human label, and is used to update the reward model.
loss = log(σ(r_j - r_k))
"j is better than k"
3 Train policy with PPO
A new post is sampled from the dataset.
The policy π generates a summary for the post.
The reward model calculates a reward for the summary.
The reward is used to update the policy via PPO.

Steps for policy creation and optimization



Reinforced learning from AI feedback (RLAIF)

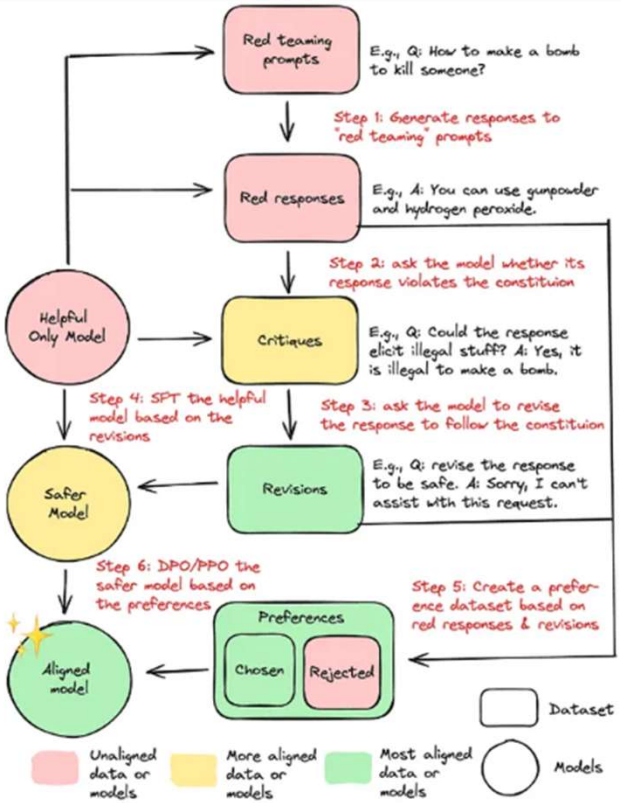


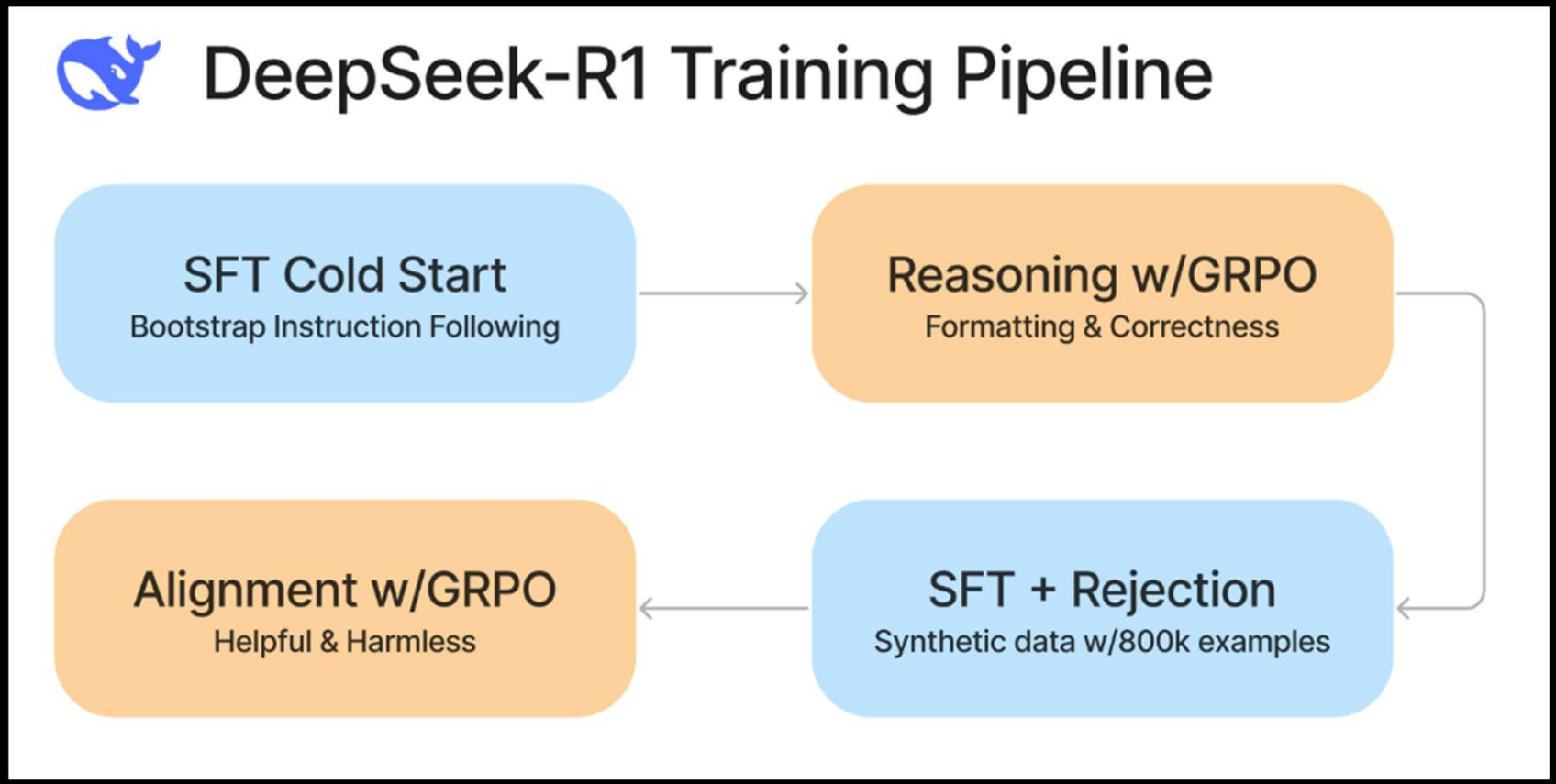
Figure 7. Reinforcement Learning from AI Feedback (RLAIF) and Constitutional AI (CAI)

Group Relative Policy Optimization (GRPO)

- Used by DeepSeek-R1
- At it’s core aimed at improving reasoning ability
  - Train the model to have reasoning traces
    <reasoning></reasoning>
  - Deterministic rewards for formatting, consistency, and correctness

https://ghost.oxen.ai/why-grpo-is-important-and-how-it-works/

Group Relative Policy Optimization (GRPO)



PPO Vs GRPO

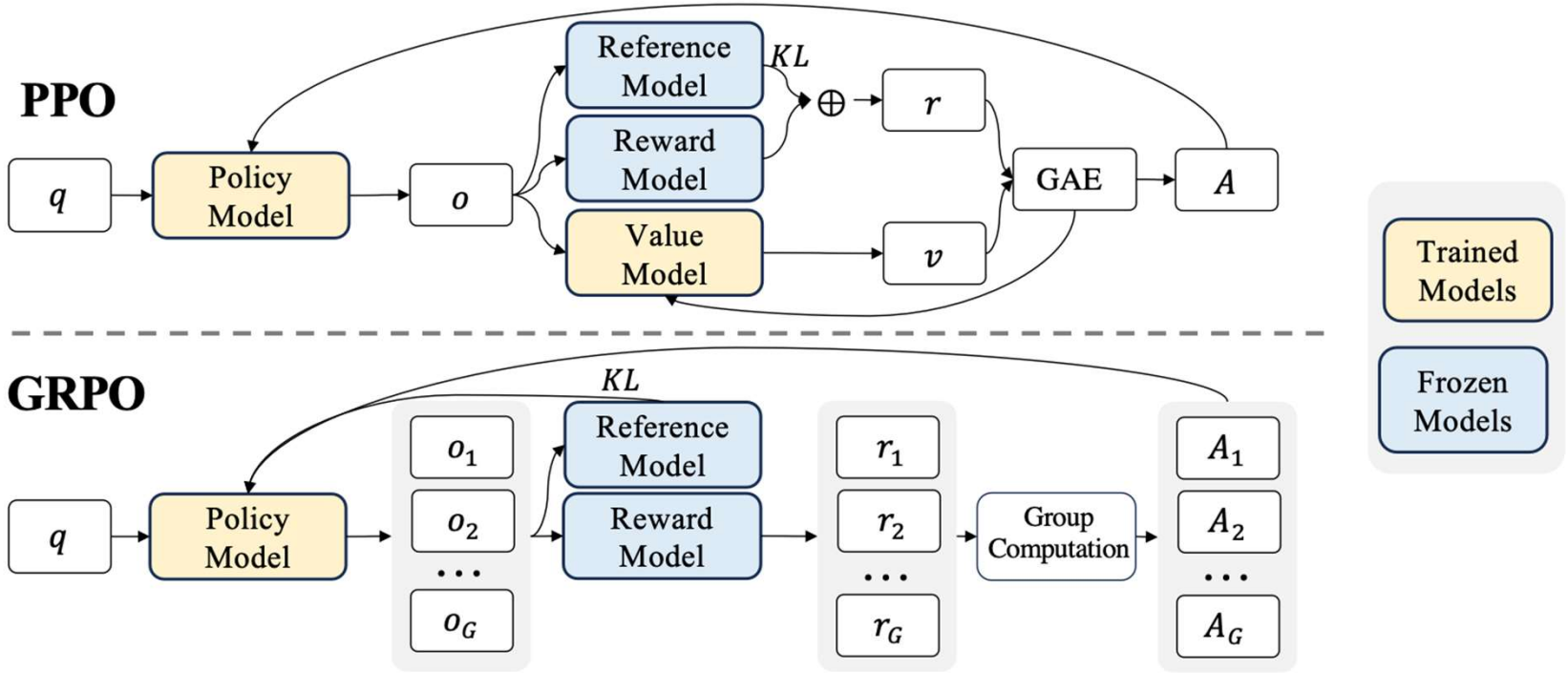
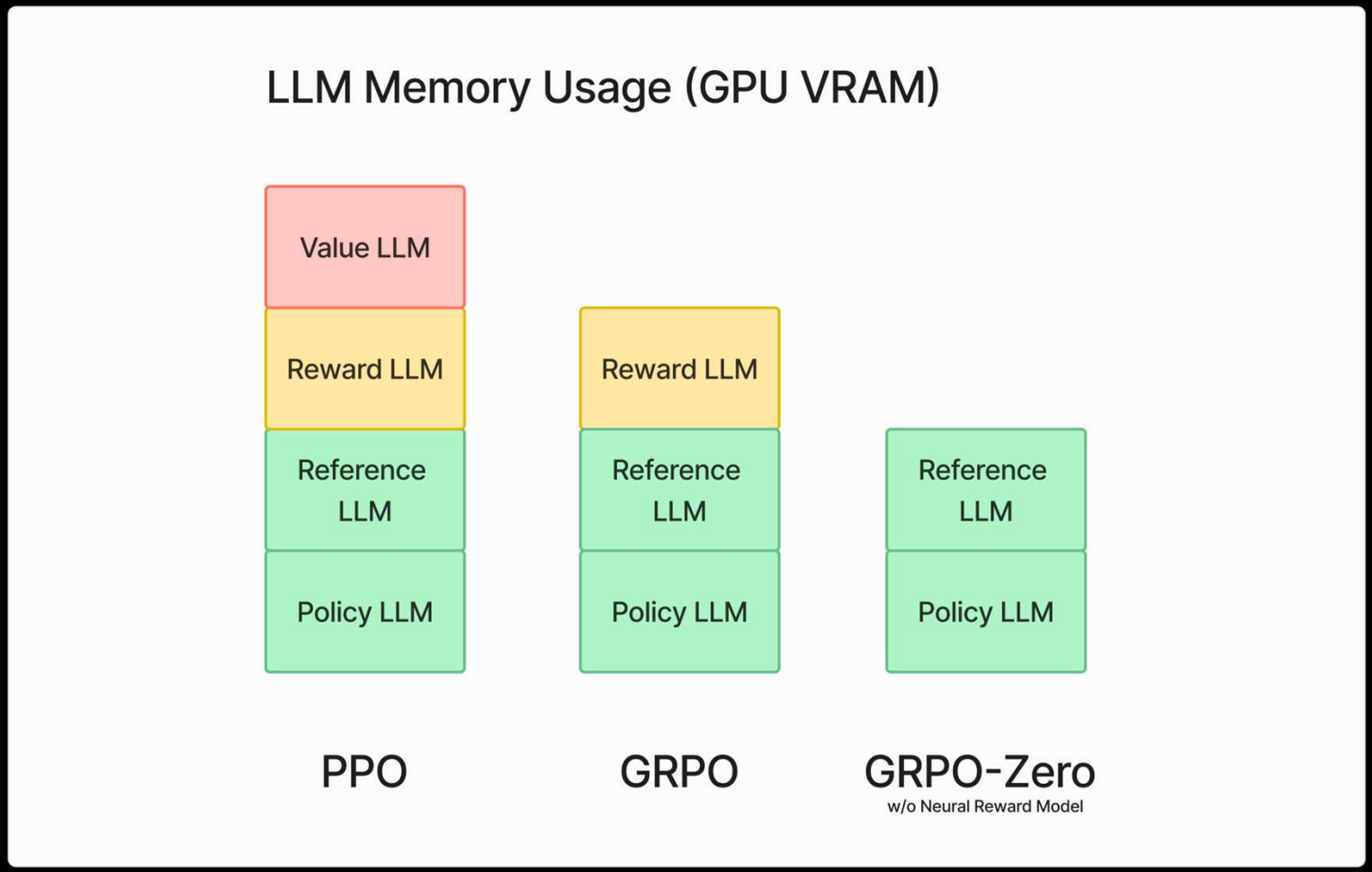
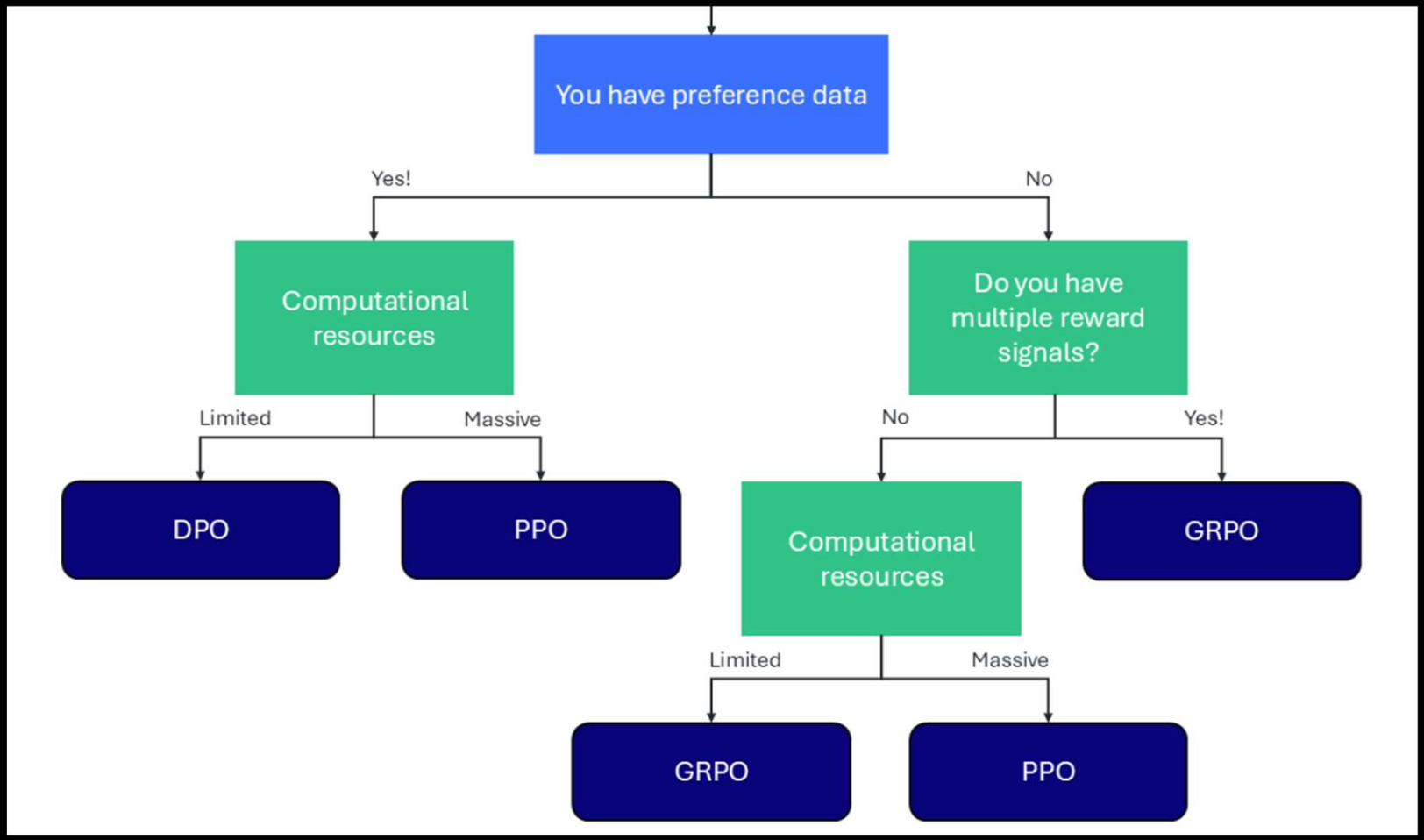


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

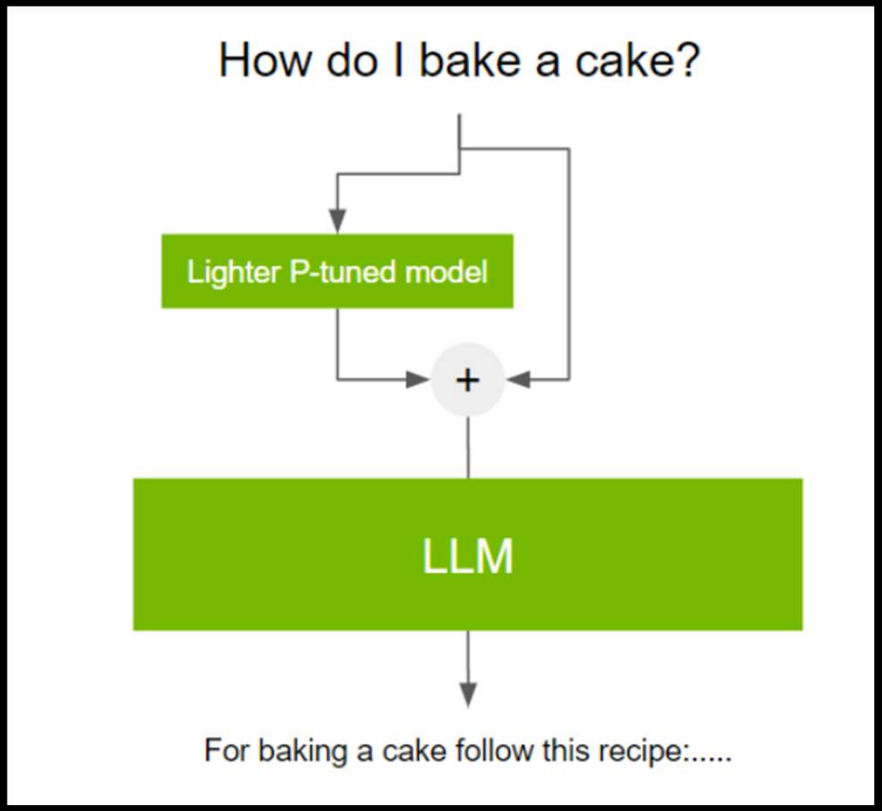
GRPO is compute and memory efficient



Choosing the right RLHF technique



Prompt tuning: P-tuning, Prefix tuning



https://ankitasinha0811.medium.com/ways-to-improve-an-llm-performance-while-keeping-the-llm-weights-frozen-7f67d740c5e2

Fine tuning models with LoRA

https://www.kaggle.com/code/nilaychauhan/fine-tune-gemma-models-in-keras-using-lora

https://dassum.medium.com/fine-tune-large-language-model-llm-on-a-custom-dataset-with-qlora-fb60abdeba07